Name: Ojas Subodh Satardekar

Student ID: 20088640

Module: Network Systems & Administration (B9IS121)

Lecturer: Prof. Kingsley Ibomo

Assessment Title: Automated Container deployment and Administration in the cloud

Project Video Link
https://drive.google.com/file/d/11L3hTtDFpLeQHrl9HVga5aki2qFOkfXZ/view?usp=sharing

Github Repo Link : ojas1216/portfolio

Contents

1. Introduction 4

2. Development Environment – Windows Subsystem for Linux (WSL) 4

3. Project Architecture 4

4. Azure Architecture……. 5

4.1 Resource Group 5

4.2 Virtual Network (VNet) 5

4.3 Subnet 5

4.4 Network Security Group (NSG) 5

4.5 Public IP and Network Interface 5

4.6 Linux Virtual Machine 5

5. Infrastructure Provisioning using Terraform 6

5.1 Terraform Workflow 6

5.2 VM Deployment Issue 6

6. Configuration Management using Ansible 8

7. Containerization using Docker 9

8. Continuous Integration and Continuous Deployment (CI/CD) 10

9. Security and Cost Management 10

10. Reflection and Learning Outcomes 10

11. Conclusion 10

12. References 11

13. Annexure 12

13.1 Annexure A – Terraform Configuration (main.tf Snippet) 12

13.2 Annexure B – Ansible Playbook Snippet (deploy.yml) 12

Table Of Figures

Fig 1.1 Terraform Init Command 6

Fig 1.2 Terraform Plan Command 7

Fig 1.3 Terraform Apply Command 7

Fig 1.4 Terraform Apply Output with Public IP 7

Fig 2.1 Ansible Playbook Execution Command 8

Fig 2.2 Ansible Playbook Successful Execution 8

Fig 3.1 Docker Build Command 11

Fig 3.2 Docker Container Run Command……..... 11

Fig 13.1 Project Architecture Diagram ……... 13

1. Introduction

Cloud computing has fundamentally transformed how to design the infrastructure, deployment and its maintenance. The traditional server provisioning relied heavily on manual configuration, which had often inconsistency, drift in configuration and increased security risks. Modern devops methods provides automation, repeatability and continuous delivery in order to reduce the operational overhead and improve deployment reliability. IaC (Infrastructure as Code) enables infrastructure to be defined programmatically, ensuring consistent and reproducible environments (Rahman et al., 2020).

The report demonstrates the design and implementation of a fully automated container deployment pipeline in Microsoft Azure. The solution shows cloud infrastructure using Terraform, configures the environment using Ansible and containerize the application using Docker and automates deployment using Github Actions. This research indicates that the devops adoption improves the frequency of deployment, its stability and overall organizational performance (Forsgren, Humble and Kim, 2021).Main objective of this project was to design a automated , secure and cost efficient cloud native deployment system.

2. Development Environment – Windows Subsystem for Linux (WSL)

Before starting anything and deploying to cloud , it was necessary to have a suitable working environment. Since many devops tools are designed to operate in Linux systems, I used Windows Subsystem for Linux i.e. WSL on my Windows laptop.

WSL allows a Linux distribution such as Ubuntu to run in windows without installing a full virtual machine. It provides interface of Linux kernel and allows execution of Linux commands. Tools like Terraform. Ansible and SSH are more stable and compatible with Linux environments .

WSL allowed me to install Terraform using Linux package repositories, install Azure CLI , generate SSH key pairs using "*ssh-keygen*" ,using linux file paths and execute Ansible playbooks.

I generated SSH key using :

ssh-keygen -t rsa -b 25519

This created a public key and private key pair. The public key was later added to Azure VM for secure authentication. It gave me a real Linux server feel, bringing experience similar to working with real industry work.

3. Project Architecture

The architecture follows a layered cloud deployment model. The infrastructure layer is provisioned using Terraform within Microsoft Azure. Networking components such as Virtual Network, Subnet and Network Security Group ensure controlled communication. The compute layer consists of a Linux Virtual Machine hosting Docker Engine. The application layer runs inside a containerised environment, ensuring portability and isolation. GitHub Actions provides an automation layer implementing Continuous Integration and Continuous Deployment (CI/CD), enabling automatic application updates upon source code changes.

4. Azure Architecture and Design

The architecture of the system followed proper cloud networking design principles. All resources were created inside an Azure Resource Group called ca-network-automation-rg.

4.1 Resource Group

A Resource Group in Azure acts as a logical container for related cloud resources. Grouping resources makes it easier to manage, monitor and delete them together.

4.2 Virtual Network (VNet)

A Virtual Network was created to allow secure communication between cloud resources. The VNet had an address space of *10.0.0.0/16*.

4.3 Subnet

Inside the VNet, a subnet was created with address range *10.0.1.0/24*. Subnets help organise resources and improve network segmentation.

4.4 Network Security Group (NSG)

The NSG acted as a firewall. Two inbound rules were defined:

- Allow SSH (Port 22)
- Allow HTTP (Port 80)

All other ports were implicitly blocked. This improved security by limiting attack surface.

4.5 Public IP and Network Interface

A Public IP address was assigned to the VM. The network interface connected the VM to the subnet and public IP.

4.6 Linux Virtual Machine

A Linux VM running Ubuntu 22.04 was deployed. The VM size selected was compatible with Gen2 hypervisor.

This architecture ensured:

- Network isolation
- Controlled access
- Proper separation of infrastructure components

5. Infrastructure Provisioning using Terraform

Terraform was used to define and deploy Azure infrastructure using code. All resources were declared inside a file called main.tf.

5.1 Terraform Workflow

The main commands used were:

- *terraform init*: terraform init downloads Azure provider plugins
- *terraform plan*: terraform plan shows what resources will be created
- *terraform apply*: terraform apply creates the resources
- *terraform destroy*: terraform destroy deletes resources

5.2 VM Deployment Issue

During deployment, I encountered a VM SKU availability error. The selected VM size was not available in the northeurope region.

I used Azure CLI to check available sizes:

az vm list-skus --location northeurope

Later, another issue occurred related to hypervisor generation mismatch. The VM size was not compatible with the selected Ubuntu image.

To fix this:

- I changed the image to *22_04-lts-gen2*
- Selected a compatible VM size

After correcting these configurations, Terraform deployment succeeded.

This experience helped me understand how cloud regions and VM compatibility affect deployment.

```
mario@Mario:~/ca-automation/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/azurerm...
- Installing hashicorp/azurerm v4.61.0...
- Installed hashicorp/azurerm v4.61.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
mario@Mario:~/ca-automation/terraform$
```

Fig 1.1 Terraform Init command

```
mario@Mario:~/ca-automation/terraform$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_linux_virtual_machine.vm will be created
  + resource "azurerm_linux_virtual_machine" "vm" {
      + admin_username                                       = "mario"
      + allow_extension_operations                           = (known after apply)
      + bypass_platform_safety_checks_on_user_schedule_enabled = false
      + computer_name                                        = (known after apply)
      + disable_password_authentication                      = true
      + disk_controller_type                                 = (known after apply)
      + extensions_time_budget                               = "PT1H30M"
      + id                                                   = (known after apply)
      + location                                             = "northeurope"
      + max_bid_price                                        = -1
      + name                                                 = "ca-vm"
      + network_interface_ids                                = (known after apply)
      + os_managed_disk_id                                   = (known after apply)
      + patch_assessment_mode                                = (known after apply)
```

Fig 1.2 Terraform plan command

```
          + disk_size_gb              = (known after apply)
          + id                        = (known after apply)
          + name                      = (known after apply)
          + storage_account_type      = "Standard_LRS"
          + write_accelerator_enabled = false
        }

      + source_image_reference {
          + offer     = "0001-com-ubuntu-server-jammy"
          + publisher = "Canonical"
          + sku       = "22_04-lts-gen2"
          + version   = "latest"
        }

      + termination_notification (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

─────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
mario@Mario:~/ca-automation/terraform$ terraform apply
```

Fig 1.3 Terraform apply command

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

azurerm_linux_virtual_machine.vm: Creating...
azurerm_linux_virtual_machine.vm: Still creating... [00m10s elapsed]
azurerm_linux_virtual_machine.vm: Creation complete after 16s [id=/subscripti
sourceGroups/ca-network-automation-rg/providers/Microsoft.Compute/virtualMach

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

public_ip_address = "20.82.161.16"
mario@Mario:~/ca-automation/terraform$
```

Fig 1.4 Terraform apply command and Public IP obtained

6. Configuration Management using Ansible

Ansible is one of the best agentless configuration management and automation tool which communicates via SSH. Ansible does not require additional software installed on managed SSH nodes beyond SSH access unlike other agent-based systems.

The Ansible playbook automated several steps of configuration: updating system packages, installing docker, start docker service , install git , cloning the portfolio repository , building the Docker image , stop the old container and then deploy the new container. Automating all these reduces manual errors and enhances the reliability of the deployment (Lwakatare et al., 2020).

The command used to run the ansible playbook is :

ansible-playbook -i inventory.ini deploy.yml

Ansible ensured repeatable server config. If the playbook is run again then it does not break the system. According to Lwakatare et al. (2020), automation tools reduce configuration errors and improve operational consistency.

Separating the infrastructure provisioning (Terraform) from the configuration management(Ansible) the project adheres to devops practices. This improves maintainability and scalability.

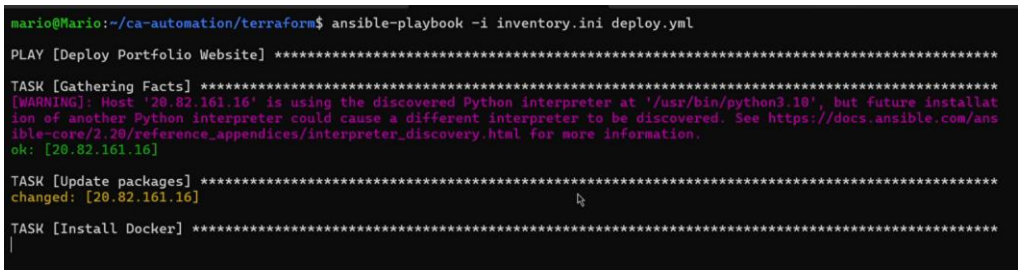

Fig 2.1 Ansible playbook command to run the playbook

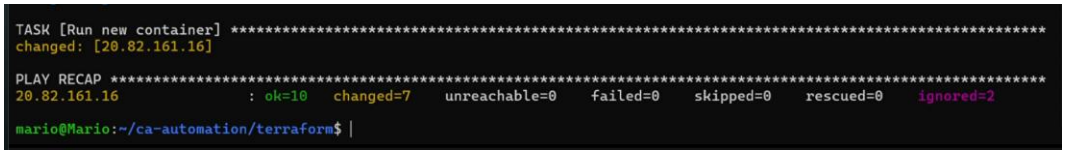

Fig 2.2 Ansible playbook executed

7. Containerization using Docker

Docker is a containerizing platform that enables applications to run in an isolated environment known as container. It keeps packages, application code and dependencies together, ensuring consistent behaviour across development and production environments.

A Dockerfile was created using Nginx base image:

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

The commands used to build the docker is :

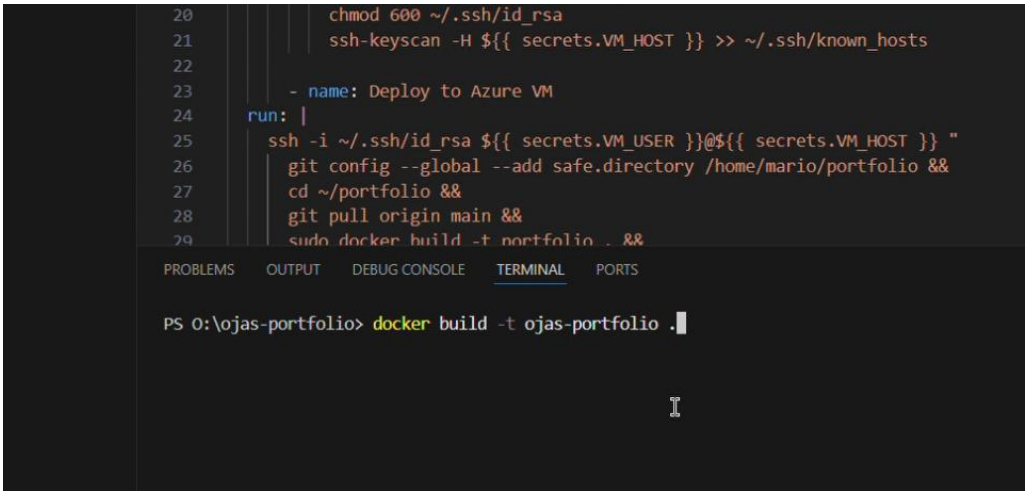

Fig 3.1 Docker Build command

The command used to build docker is :

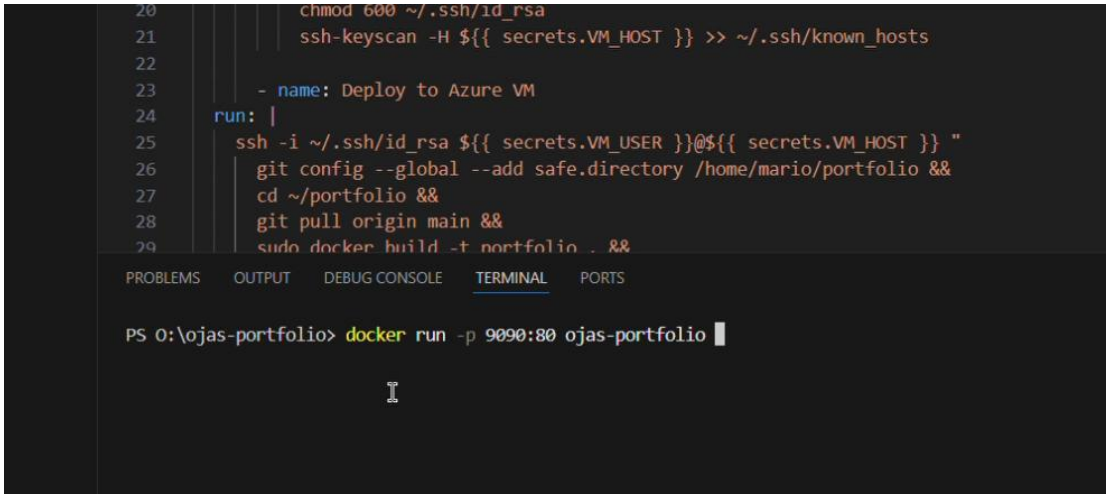

Fig 3.2 Docker Run command

Port mapping exposed the containers internal port to Azure's public interface, allowing external access to website.
Containerisation does improve portability and scalability but misconfigured containers may introduce security vulnerabilities (Sharma et al., 2023). Controlled port access and minimal exposure were implemented to reduce attack surface.

8. Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration and Continuous Deployment automate the build and deployment lifecycle of applications. GitHub Actions was configured to trigger deployment workflows upon pushes to the main branch. So whenever the changes were pushed to the main branch the GitHub workflow is triggered , next SSH connection to Azure is established , after that latest code is pulled , docker image is rebuilt and finally the container is restarted . sensitive credentials were stored in GitHub Secrets.

CI/CD pipelines improve deployment speed and reduce manual intervention. According to Zhang et al. (2022), secure CI/CD implementation prevents credential exposure.

9. Security and Cost Management

Security controls implemented:

- SSH key authentication
- Limited NSG rules
- Secure GitHub secrets
- No unnecessary services

Cloud misconfiguration remains a major security risk (ENISA, 2023). Therefore, infrastructure was carefully configured.
Cost monitoring was conducted using Azure's Cost Management dashboard. The project demonstration, all infrastructure resources were destroyed to prevent unnecessary credit usage. Responsible cloud resource management is an essential component of modern cloud engineering.

10. Reflection and Learning outcomes

This project improved my understanding of cloud automation significantly.

I learned:

- How Infrastructure as Code improves repeatability
- How networking works in Azure
- How to troubleshoot VM deployment issues
- How configuration management tools simplify server setup
- How CI/CD automates deployments

Using WSL gave me confidence working in Linux environments.

11. Conclusion

This project successfully demonstrated a secure and automated Azure-based container deployment pipeline. By integrating Terraform, Ansible, Docker and GitHub Actions, a reproducible and scalable infrastructure solution was achieved. The layered design ensured modularity, while automation reduced manual configuration errors. Challenges encountered during VM provisioning provided valuable practical experience in troubleshooting cloud infrastructure compatibility issues. The final

implementation aligns with industry standards for DevOps, Infrastructure as Code and CI/CD security. The project highlights the importance of automation, security awareness and cost governance in modern cloud-native systems

12. References

Rahman, A., Williams, L. and Zhang, H. (2020) 'An empirical study of Infrastructure as Code practices and challenges', *Empirical Software Engineering*. Available at: https://doi.org/10.1007/s10664-020-09839-7 (Accessed: 20 February 2026).

Forsgren, N., Humble, J. and Kim, G. (2021) *Accelerate: The Science of Lean Software and DevOps*. Portland: IT Revolution Press.

Hassan, A., Adams, B. and Jiang, Z.M. (2022) 'Infrastructure as Code compliance and security analysis', *IEEE Cloud Computing*. Available at: https://ieeexplore.ieee.org (Accessed: 20 February 2026).

Lwakatare, L.E., Kuvaja, P. and Oivo, M. (2020) 'DevOps in practice: A multiple case study of five companies', *Information and Software Technology*, 114, pp. 217–230. Available at: https://doi.org/10.1016/j.infsof.2019.01.007 (Accessed: 20 February 2026).

Pahl, C. (2020) 'Containerization and the PaaS Cloud', *IEEE Cloud Computing*, 2(3), pp. 24–31. Available at: https://ieeexplore.ieee.org/document/7113784 (Accessed: 20 February 2026).

Sharma, P., Chen, Y. and Sheth, A. (2023) 'Security analysis of container-based cloud applications', *IEEE Access*, 11, pp. 21543–21560. Available at: https://ieeexplore.ieee.org (Accessed: 20 February 2026).

Zhang, X., Lee, S. and Kim, D. (2022) 'Security challenges in CI/CD pipelines', *Journal of Systems and Software*, 188, 111254. Available at: https://doi.org/10.1016/j.jss.2022.111254 (Accessed: 20 February 2026).

ENISA (2023) *Cloud Security Threat Landscape 2023*. European Union Agency for Cybersecurity. Available at: https://www.enisa.europa.eu/publications/cloud-security-threat-landscape (Accessed: 20 February 2026).

HashiCorp (2026) *Terraform Documentation*. Available at: https://developer.hashicorp.com/terraform/docs (Accessed: 20 February 2026).

Microsoft (2026) *Microsoft Azure Virtual Machines Documentation*. Available at: https://learn.microsoft.com/en-us/azure/virtual-machines/ (Accessed: 20 February 2026).

Docker Inc. (2026) *Docker Documentation*. Available at: https://docs.docker.com (Accessed: 20 February 2026).

Red Hat (2026) *Ansible Documentation*. Available at: https://docs.ansible.com (Accessed: 20 February 2026).

GitHub (2026) *GitHub Actions Documentation*. Available at: https://docs.github.com/en/actions (Accessed: 20 February 2026).

13. Annexure

13.1 Annexure A – Terraform Configuration (main.tf Snippet)

```
provider "azurerm" {
features {}
}

resource "azurerm_resource_group" "rg" {
name     = "ca-network-automation-rg"
location = "northeurope"
}

resource "azurerm_virtual_network" "vnet" {
name              = "ca-vnet"
address_space      = ["10.0.0.0/16"]
location           = azurerm_resource_group.rg.location
resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_linux_virtual_machine" "vm" {
name              = "ca-vm"
size              = "Standard_D2as_v7"
admin_username     = "mario"
disable_password_authentication = true
}
```

13.2 Annexure B – Ansible Playbook Snippet (deploy.yml)

```
---
- name: Deploy Portfolio Website
  hosts: web
  become: yes

  tasks:

    - name: Update system packages
      apt:
        update_cache: yes

    - name: Install Docker
      apt:
        name: docker.io
        state: present

    - name: Start Docker service
      service:
        name: docker
        state: started
        enabled: yes

    - name: Install Git
      apt:
        name: git
        state: present

    - name: Clone Portfolio Repository
      git:
```

```
    repo: https://github.com/yourusername/portfolio.git
    dest: /home/mario/portfolio
    version: main

  - name: Build Docker Image
    command: docker build -t portfolio /home/mario/portfolio

  - name: Stop existing container (if running)
    command: docker stop portfolio
    ignore_errors: yes

  - name: Remove existing container (if exists)
    command: docker rm portfolio
    ignore_errors: yes

  - name: Run Docker Container
    command: docker run -d -p 80:9090 --name portfolio portfolio
```

The above playbook automates server configuration by installing required dependencies, cloning the application repository, building the Docker image and deploying the container. The playbook ensures repeatable and consistent server setup.

13.3 Annexure C – Project Architecture

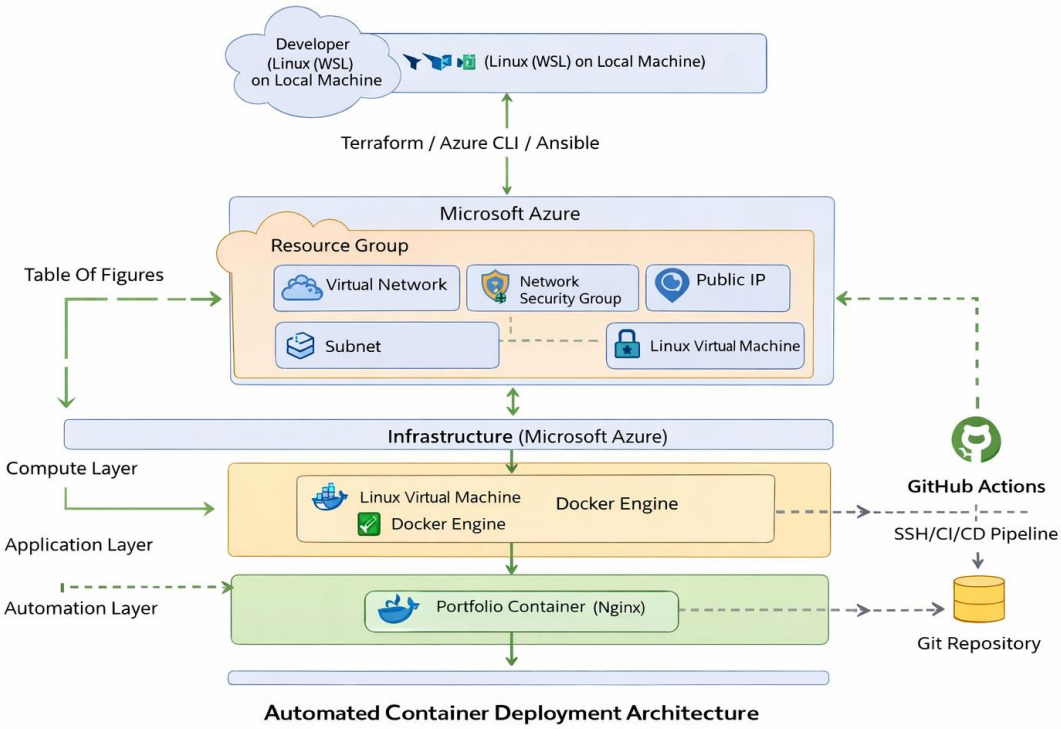

Figure 13.1: Project Architecture Diagram